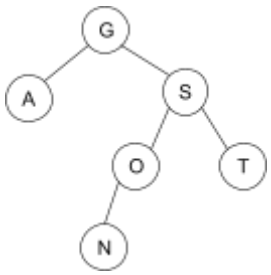
American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

PROBLEM: The *Binary Search Tree* built from the letters **GASTON** is shown on the right. There are various ways to implement this data structure. A straightforward approach—but wasteful of space—is to use a 1D array and store the root at position 0. The left child of the root is at position 1 and the right child is at position 2. In general, the node at position *i* stores its children in positions 2*i*+1 and 2*i*+2. For example, the tree on the right would be represented with the following array:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	..
G	A	S			O	T					N			

Although a serious drawback of this representation is that the resulting array is *sparse*, meaning that there will be many unused locations, this method can be implemented without using advanced programming concepts. After using this representation to build a binary search tree, your program will determine the number of empty spaces between each of the letters in the array.

EXAMPLE:

Input	`GASTON`
Output	`0 0 2 0 4`
Explanation	The letter **G** is placed in location 0. The letter **A** is the left child of **G** so it is placed in location 2*0+1 = 1. The letter **S** is the right child of **G** so it is placed in location 2*0+2 = 2. The letter **T** is the right child of **S** so it is placed in location 2*2+2 = 6. The letter **O** is the left child of **S** so it is placed in location 2*2+1 = 5. The letter **N** is the left child of **O** so it is placed in location 2*5+1 = 11. The output is the string containing the number of empty locations between each of the letters which is `0 0 2 0 4`.

INPUT: Input a single string of uppercase letters that is no more than 20 characters in length.

OUTPUT: Output a string of numbers representing the number of empty spaces between the letters in the array that represents the binary search tree created by the input, each number separated by a single space.

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

SAMPLE INPUT:

1. GASTON
2. DUMBO
3. SIMBA
4. PETERPAN
5. RAPUNZEL
6. PINOCCHIO

SAMPLE OUTPUT:

1. 0 0 2 0 4
2. 0 0 2 6
3. 0 1 0 2
4. 0 0 0 0 0 1 1
5. 0 0 1 1 2 9 20
6. 0 1 0 2 0 1 7 2

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

TEST DATA

TEST INPUT:

1. ARIEL
2. JASMINE
3. CINDERELLA
4. BUZZLIGHTYEAR
5. SLEEPINGBEAUTY
6. AA

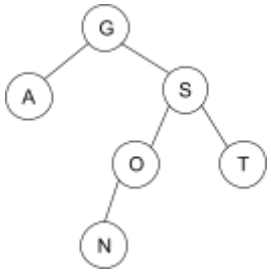
TEST OUTPUT:

1. 1 2 5 0
2. 0 0 1 0 3 2
3. 0 0 2 0 5 0 0 10 1
4. 0 0 2 0 4 0 0 9 1 1 19 0
5. 0 0 0 0 0 0 0 0 0 5 1 13 0
6. 0

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

PROBLEM STATEMENT:



The *Binary Search Tree* built from the letters **GASTON** is shown on the right. There are various ways to implement this data structure. A straightforward approach—but wasteful of space—is to use a 1D array and store the root at position 0. The left child of the root is at position 1 and the right child is at position 2. In general, the node at position *i* stores its children in positions 2*i*+1 and 2*i*+2. For example, the tree on the right would be represented with the following array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	..
G	A	S			O	T					N			

Although a serious drawback of this representation is that the resulting array is *sparse*, meaning that there will be many unused locations, this method can be implemented without using advanced programming concepts. After using this representation to build a binary search tree, your program will determine the number of empty spaces between each of the letters in the array.

EXAMPLE:

Input:

```
GASTON
```

Output:

```
0 0 2 0 4
```

Explanation:

The letter **G** is placed in location 0.
The letter **A** is the left child of **G** so it is placed in location 2*0+1 = 1.
The letter **S** is the right child of **G** so it is placed in location 2*0+2 = 2.
The letter **T** is the right child of **S** so it is placed in location 2*2+2 = 6.
The letter **O** is the left child of **S** so it is placed in location 2*2+1 = 5.
The letter **N** is the left child of **O** so it is placed in location 2*5+1 = 11.

The output is the string containing the number of empty locations between each of the letters which is `0 0 2 0 4`.

TASK:

Complete the function **buildTree**

- The function has 1 parameter: a string of uppercase letters, *word*, that is used to create a binary search tree in a one-dimensional array
- The function returns a string of numbers, each separated by a single space, representing the number of empty array locations between each of the letters

You may create additional functions that are called from **buildTree** if needed in solving the problem.

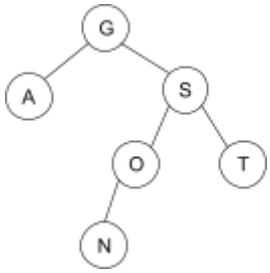
CONSTRAINTS:
Each string will be no more than 20 characters. Use the ACSL definition for building the tree.

DATA PROVIDED:
There are 6 sets of Sample Data for debugging and 6 sets of Test Data for scoring. The test cases will vary in difficulty. You should create sample data of your own to fully test your program.

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

问题 ：由字符串 GASTON **构建的*二叉搜索树*如右图所示。该数据结构有多种实现方式。一种直观但较占空间的方法是使用一维数组，并将根节点存储在位置** 0**。根节点的左子节点位于位置** 1**，右子节点位于位置** 2**。一般而言，位于位置** i **的节点将其子节点存储在位置**$2i+1$ **和** $2i+2$ **。例如，右侧的树将用以下数组表示：**



0	1	2	3	4	5	6	7	8	9	10	11	12	13	..
G	A	S			O	T					N			

尽管**这种表示方**法**存在明显缺陷，生成的数组会包含大量*空位*，即许多未被占用的数组单元，但**该方法的实现无需使用高级编程概念。**当采用此方法构建二叉搜索树后，程序需要计算数组中**每个**字母之间的空位数量。**

示例：

输入	GASTON
输出	0 0 2 0 4
解析	**字母 G 被放置在位置** 0。 **字母 A 作为 G 的左子节点，被放置在位置** 2*0+1 = 1。 **字母 S 作为 G 的右子节点，被放置在位置** 2*0+2 = 2。 **字母 T 作为 S 的右子节点，被放置在位置** 2*2+2 = 6。 **字母 O 作为 S 的左子节点，被放置在位置** 2*2+1 = 5。 **字母 N 作为 O 的左子节点，被放置在位置** 2*5+1 = 11。 **输出为一个字符串，包含数组中**每个**字母间的空位数量：**0 0 2 0 4。

输入：输入一个由大写字母组成的字符串，其长度不超过 20 **个字符。**

输出：输出一串数字，表示输入**构建**的**二叉搜索树的数组中字母之间的空位数量，每个数字间用单个空格分隔。**

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

样例输入：

1. GASTON
2. DUMBO
3. SIMBA
4. PETERPAN
5. RAPUNZEL
6. PINOCCHIO

样例输出：

1. 0 0 2 0 4
2. 0 0 2 6
3. 0 1 0 2
4. 0 0 0 0 0 1 1
5. 0 0 1 1 2 9 20
6. 0 1 0 2 0 1 7 2

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

测试数据

测试输入：

1. ARIEL
2. JASMINE
3. CINDERELLA
4. BUZZLIGHTYEAR
5. SLEEPINGBEAUTY
6. AA

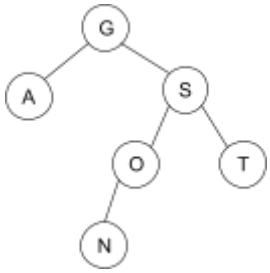
测试输出：

1. 1 2 5 0
2. 0 0 1 0 3 2
3. 0 0 2 0 5 0 0 10 1
4. 0 0 2 0 4 0 0 9 1 1 19 0
5. 0 0 0 0 0 0 0 0 0 5 1 13 0
6. 0

American Computer Science League

2024-2025 ● Contest 4: Binary Tree Arrays ● Junior Division

问题陈述：



由字符串 GASTON **构建的二叉*搜索树*如右图所示。该数据结构有多种实现方式。一种直观但较占空间的方法是使用一维数组，并将根节点存储在位置** 0**。根节点的左子节点位于位置** 1**，右子节点位于位置** 2**。一般而言，位于位置** i **的节点将其子节点存储在位置** $2i+1$ **和** $2i+2$**。例如，右侧的树将用以下数组表示：**

0	1	2	3	4	5	6	7	8	9	10	11	12	13	..
G	A	S			O	T					N			

尽管**这种表示方**法**存在明显缺陷，生成的数组会包含大量*空位*，即许多未被占用的数组单元，但**该方法的实现**无需**使用**高级编程**概念。**当采用此方法构建二叉搜索树后，程序需要计算数组中**每个**字母之间的空位数量。**

示例：

输入：

```
GASTON
```

输出：

```
0 0 2 0 4
```

解析：

字母 G 被放置在位置 0。
字母 **A** 作为 **G 的左子节点，被放置在位置** 2*0+1 = 1。
字母 **S** 作为 **G 的右子节点，被放置在位置** 2*0+2 = 2。
字母 **T** 作为 **S 的右子节点，被放置在位置** 2*2+2 = 6。
字母 **O** 作为 **S** 的左子节点，被放置在位置 2*2+1 = 5。
字母 **N** 作为 **O** 的左子节点，被放置在位置 2*5+1 = 11。

输出为一个字符串，包含数组中每个**字母间的空位数量：**0 0 2 0 4。

任务：

完成函数 buildTree

- **该函数有** 1 **个参数：由大写字母组成的字符串，***word***，用于在一维数组中创建二叉搜索树。**
- **该函数返回一个由数字组成的字符串，各数字间用单个空格分隔，表示数组中**每个**字母之间的空位数量。**

如有需要，你可以构建额外的函数供 **buildTree** 调用来解答此问题。

约束条件：
每个字符串长度不超过 20 **个字符。使用** ACSL **定义的规则构建二叉树。**

数据说明：
包含 6 **组用于调试的**样例**数据和** 6 **组用于评分的测试数据。测试案例难度各不相同。**
您应该自行创建样例**数据以充分验证程序。**